

Otimização de Multiplicação de Matrizes com OpenMP: vetorização, granularidade e padrão de acesso à memória

Trabalho Final — Software de Alto Desempenho (INF01094)

João Vitor do Amaral Spolavore

Instituto de Informática — UFRGS

2026/1

Problema

- Multiplicação de matrizes densas $C = A \times B$, dupla precisão, $N = 1024$;
- $2N^3 \approx 2,15$ GFLOP por execução; 24 MiB de dados;
- núcleo de álgebra linear, simulação e ML; reuso de dados e paralelismo abundante.

Ambiente: PCAD/UFRGS, nó hype5

- 2× Xeon E5-2650 v3 (Haswell): **20 cores / 40 threads HT, 2 sockets NUMA**;
- 128 GB DDR4; L3 25 MiB/socket;
- governador de frequência fixo em performance.

Pergunta da investigação

Quais intervenções OpenMP produzem ganho real, quais não produzem, e *por quê*?

v0: laços i-j-k canônicos

```
for (i = 0; i < n; i++)  
  for (j = 0; j < n; j++) {  
    double sum = 0.0;  
    for (k = 0; k < n; k++)  
      sum += A[i*n+k] * B[k*n+j];  
    C[i*n+j] = sum;  
  }
```

Baseline: 2,298 s ($\sigma = 0,0003$)

Metodologia

- **sem profiler**: tempo medido com `omp_get_wtime()` em volta do kernel;
- 1 warm-up não medido + **5 repetições**; mediana $\pm$ desvio ($< 1\%$);
- threads fixadas nos cores físicos (`OMP_PROC_BIND/OMP_PLACES`);
- mesmas entradas em todas as versões;
- **validação**: comparação elemento a elemento com a v0 — diferença **zero** nas 4 versões.

- Pico teórico do nó ≈ 736 GFLOP/s; a v0 atinge **0,94 GFLOP/s** $\approx$ **0,1%** $\rightarrow$ o gargalo **não** é processamento.
- Laço interno lê $B[k*n+j]$ com **stride de 8 KiB**: cada iteração toca uma linha de cache nova e usa só 1 dos 8 doubles trazidos.
- Sintoma do mapa da disciplina: *tempo dominado por memória / acessos com stride*.

v1 — omp simd no laço interno (sem ganho)

```
double sum = 0.0;
#pragma omp simd reduction(+:sum)
for (k = 0; k < n; k++)
    sum += A[i*n+k] * B[k*n+j];
```

- **Hipótese:** SIMD processa 4 doubles por instrução (AVX2) $\rightarrow \sim 4\times$.

- O compilador **vetoriza de fato**, mas monta os vetores de B com cargas escalares — o acesso tem stride;
- o tráfego de memória por iteração **não muda**.

Resultado medido

2,388 s $\rightarrow$ **0,96** $\times$ (leve piora)

v2 e v3 — a MESMA técnica, granularidades opostas

v2: laço externo (grossa)

```
#pragma omp parallel for \  
    schedule(static)  
for (i = 0; i < n; i++)  
    ... /* corpo igual ao da v0 */
```

- 1 região paralela; N/p linhas por thread, sem sincronização interna.

v3: laço interno (fina)

```
for (i ...) for (j ...) {  
    #pragma omp parallel for \  
        reduction(+:sum)  
    for (k = 0; k < n; k++) ...  
}
```

- Região paralela criada $N^2 \approx 10^6$ vezes; reduction + **barreira entre 2 sockets** a cada vez.

v2 e v3 — resultados (20 threads)

v2 — laço externo (grossa)

0,132 s → **17,5×**

v3 — laço interno (fina)

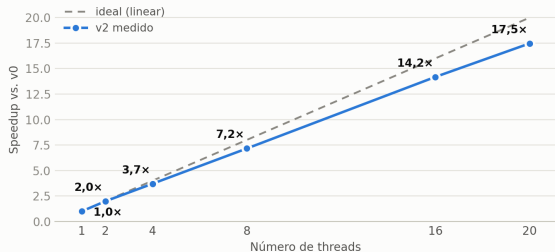
10,193 s → **0,23×**

Baseline v0 (1 thread): **2,298 s** — com os mesmos 20 cores, a v3 ficou **4,4×** mais lenta que o sequencial.

Escalabilidade da v2

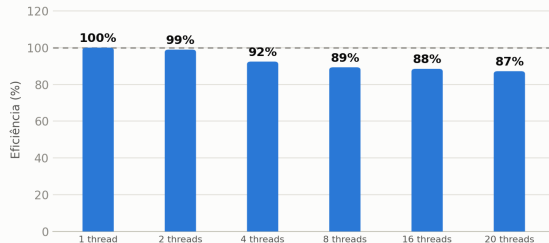
Escalabilidade da v2: speedup vs. número de threads

Matmul N=1024 — pontos acima da linha ideal indicam speedup superlinear (ver discussão)



Eficiência paralela da v2

eficiência(p) = speedup(p) / p, relativa à v2 com 1 thread — acima de 100% = superlinear



Quase linear até 20 threads (99% → 87%); a queda gradual acompanha a fronteira NUMA.

v4 — troca de laços i-k-j (ataca a causa raiz)

```
#pragma omp parallel for \  
    schedule(static)  
for (i = 0; i < n; i++) {  
    for (j ...) C[i*n+j] = 0.0;  
    for (k = 0; k < n; k++) {  
        const double a = A[i*n+k];  
        for (j = 0; j < n; j++)  
            C[i*n+j] += a * B[k*n+j];  
    }  
}
```

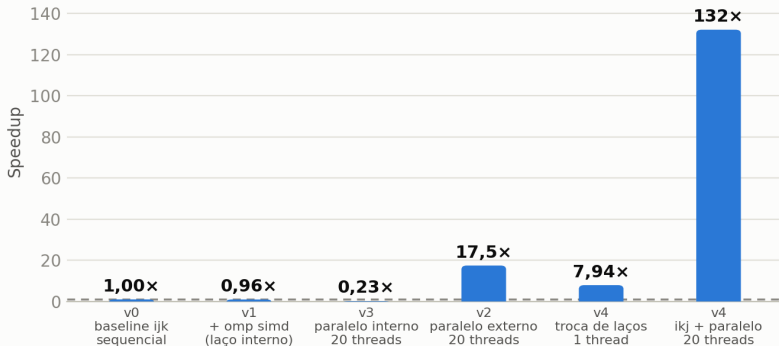
- Laço interno percorre B e C com **stride 1**: vetorização efetiva, prefetch sequencial, linha de cache usada por inteiro;
- soma na mesma ordem de $k \rightarrow$ resultado **bit a bit idêntico** ao da v0;
- combina a correção do padrão de acesso com o paralelismo da v2.

Resultados (hype, $N = 1024$, mediana de 5 repetições)

Versão	Configuração	Tempo	Speedup	GFLOP/s	Efic.
v0	sequencial	2,298 s	1,00×	0,94	—
v1	omp simd, sequencial	2,388 s	0,96 ×	0,90	—
v3	paralelo interno, 20 threads	10,193 s	0,23 ×	0,21	1%
v2	paralelo externo, 20 threads	0,132 s	17,47×	16,3	87%
v4	i-k-j (troca de laços), 1 thread	0,289 s	7,9×	7,4	—
v4	i-k-j + paralelo, 20 threads	0,017 s	132 ×	123,5	83%

Speedup por versão (vs. baseline v0)

Matmul N=1024 — maior é melhor; linha tracejada marca o baseline (1x)



Discussão: por que cada resultado aconteceu

- **v1 (0,96×)**: SIMD multiplica a capacidade de cálculo, mas o tempo é gasto esperando memória. *Uma técnica só é boa otimização quando resolve o gargalo dominante.*
- **v3 (0,23×)**: 10^6 regiões paralelas $\Rightarrow$ fork/join + barreiras **atravessando 2 sockets NUMA** custam mais que o trabalho útil.
- **v2 (17,5×, 87%)**: granularidade grossa escala quase linear; a eficiência cai suavemente ao ocupar o segundo socket (acessos NUMA remotos).
- **v4 (132×)**: o par v1/v4 fecha a prova — as mesmas unidades SIMD rendem 0,96× com stride e $7,9\times$ (1 thread) com acesso corrigido; com 20 threads, 123,5 GFLOP/s.

Obrigado